

Architecture Microservices

Du Monolithe aux Microservices

OFPPT · Développement Digital — Option Full Stack

■ 1. Architecture Monolithique

Définition

Une application monolithique est développée **en un seul bloc**, avec une même technologie, et déployée dans un seul serveur d'application. Tous les composants (interface, logique métier, accès aux données) forment une seule unité de déploiement (WAR, JAR, EAR, DLL...).

Structure — Exemple e-commerce

Couche	Rôle	Exemple (e-commerce)
Interface Utilisateur	Présentation	Pages web, formulaires
Logique Métier	Traitements	Commandes, paiements, clients
Accès aux Données	Persistance	Base de données unique partagée

Problèmes principaux

- **Déploiement compliqué** : Tout changement oblige à redéployer l'application entière. Un bug dans un module peut faire tomber tout le système.
- **Scalabilité non optimisée** : On duplique toute l'application même si seul un module est surchargé (ex. : fort trafic sur le module Produit).
- **Code difficile à maintenir** : La base de code grossit, les équipes sont interdépendantes, changer de technologie est très coûteux.

■ 2. Architecture Microservices

Définition

L'architecture Microservices décompose une application en **petits services indépendants**, chacun exécutant une fonction métier précise, possédant sa propre base de données, pouvant être déployé de façon autonome et communiquant via des **APIs REST**.

Principe fondamental

- Chaque service est **responsable d'une seule fonctionnalité**.
- Chaque service peut être **développé, déployé et mis à l'échelle indépendamment**.
- Un dysfonctionnement d'un service n'arrête pas les autres (**résilience**).

Exemple : e-commerce en Microservices

Service	Fonctionnalité	Base de données
Service Client	Gestion des comptes	MongoDB
Service Produit	Catalogue produits	MySQL
Service Commande	Gestion commandes	PostgreSQL
Service Paiement	Traitement paiements	MySQL

■ 3. Concepts Clés des Microservices

■ Autonomie technologique

Chaque service choisit librement son langage (Node.js, Java, Python...), son framework, son serveur et sa base de données. Les services sont généralement conteneurisés avec **Docker**.

■ Déploiement ciblé

Chaque service a sa propre version (ex. Service Client v2.0, Service Produit v10.2). Les mises en production sont indépendantes : moins de coordination, moins de risques.

■ Scalabilité ciblée

Pendant le Black Friday, on peut multiplier uniquement les instances des services Produit et Commande, sans toucher aux autres (Scale UP ciblé).

Avantages vs Inconvénients

✓ Avantages	✗ Inconvénients
✓ Déploiement indépendant	✗ Communication réseau complexe
✓ Scalabilité ciblée	✗ Plusieurs bases de données à gérer
✓ Technologies multiples	✗ Tests et débogage distribués
✓ Résilience améliorée	✗ Infrastructure plus coûteuse
✓ Équipes autonomes	✗ Latence des appels distants

■ 4. Communication entre Services

Mode	Description	Couplage	Technologies	Exemple
Synchrone	Appel direct — attend la réponse	Fort	HTTP/REST, gRPC	Vérification stock
Asynchrone (point/point)	Envoie un message et continue	Moyen	RabbitMQ, Kafka, Amazon SQS	Envoi d'email
Diffusion d'événements	Publie un event — plusieurs abonnés reçoivent	Faible	Kafka, Event Bus	Notifications multi-services

■ 5. API Gateway

L'**API Gateway** est le **point d'entrée unique** entre les clients et les microservices. Elle centralise le routage, l'authentification (JWT), le load balancing et le rate limiting.

Technologie	Description
Kong	Open source, basé sur Nginx — très populaire
AWS API Gateway	Service managé Amazon Web Services
Nginx	Serveur web configurable, haute performance
Express Gateway	Basé sur Node.js, idéal pour les projets JS

■ 6. Étude de Cas — Gestion de Bibliothèque

Services identifiés

Service	Type	Fonctionnalité
Service Livre	Métier	CRUD livres — GET/POST/PUT/DELETE /api/livres
Service Client	Métier	Gestion des comptes utilisateurs
Service Emprunt	Métier	Gestion des emprunts et retours
Notification	Utilitaire	Envoi emails/SMS de rappel (async)
Paielement	Utilitaire	Facturation des frais de retard

API du Service Livre

Fonctionnalité	Méthode	Endpoint	Codes HTTP
Obtenir un livre	GET	/api/livres/:id	200, 404
Ajouter un livre	POST	/api/livres	201, 400
Modifier un livre	PUT	/api/livres/:id	200, 404
Supprimer un livre	DELETE	/api/livres/:id	200, 404

■ 7. Synthèse Comparative

Critère	Monolithique	Microservices
Déploiement	Tout ensemble	Indépendant par service
Scalabilité	Globale (tout ou rien)	Ciblée (service par service)
Technologies	Une seule stack	Multiples (polyglotte)

Base de données	Partagée unique	Une par service
Complexité	Simple au départ	Distribuée, plus complexe
Idéal pour	MVP / Petite équipe	Grande équipe / Fort trafic

■ Choisir Monolithique quand...	■ Choisir Microservices quand...
• Petite équipe (< 5 devs) • MVP / Prototype • Déploiements rares • Budget limité	• Grande équipe (> 10 devs) • Scalabilité requise • Déploiements fréquents • Tolérance aux pannes critique

■ Points Clés à Retenir

1. Monolithique = une seule unité de déploiement
2. Microservices = plusieurs services indépendants avec chacun sa propre BDD
3. Communication via REST (synchrone) ou Message Queue (asynchrone)
4. Autonomie technologique : chaque service choisit son stack (Docker)
5. Scalabilité ciblée : on scale uniquement le service surchargé
6. API Gateway = point d'entrée unique (routage, auth, load balancing)
7. Entreprises utilisatrices : Netflix, Amazon, Uber, Spotify, eBay